

Q7) Combined Volume Comparison

Create one container using a **Named Volume**, one using an **Anonymous Volume**, and one using a **Bind Mount**. Store different files in each location, remove the containers, recreate them, and verify which data persists. Compare the persistence behavior of all three storage methods and document the results.

```
# Create named volume
docker volume create student_data

# Named Volume Container
docker run -dit --name named_con -v student_data:/data ubuntu bash
docker exec named_con sh -c "echo NAMED > /data/named.txt"

# Anonymous Volume Container
docker run -dit --name anon_con -v /data ubuntu bash
docker exec anon_con sh -c "echo ANON > /data/anon.txt"

# Bind Mount Container
mkdir bind_data
echo BIND > bind_data/bind.txt

docker run -dit --name bind_con -v ${PWD}/bind_data:/data ubuntu bash

# Remove containers
docker rm -f named_con anon_con bind_con

# Recreate containers
docker run -dit --name named_con2 -v student_data:/data ubuntu bash
docker run -dit --name anon_con2 -v /data ubuntu bash
docker run -dit --name bind_con2 -v ${PWD}/bind_data:/data ubuntu bash

# Verify data
docker exec named_con2 ls /data
docker exec anon_con2 ls /data
docker exec bind_con2 ls /data
```

Q8) Docker Network, Environment Variables, Named Volume and Container Recovery

Create a custom Docker network named **student_net**. Launch a MySQL container named **mysql_db** on this network using environment variables to configure the database name, username, and password. Attach a **named volume** to persist the database data. Verify that the database is running and create a table containing your PRN number and name.

Launch a second container named **app_client** on the same network and verify communication with the MySQL container using the container name as the hostname. Insert and retrieve records from the database through the second container.

Remove the **app_client** container and recreate it on the same network. Verify that it can still communicate with the MySQL container and access the stored records.

Next, remove the **mysql_db** container without deleting the named volume. Create a new MySQL container named **mysql_db_new** using the same network configuration, environment variables, and named volume. Verify that the previously stored database and records are automatically restored from the volume.

Finally, launch another client container on the same network, establish communication with **mysql_db_new**, and verify that all previously stored data remains accessible. Display the network configuration, volume details, running containers, and database records to confirm successful recovery and communication

—

Create custom network

```
docker network create student_net
```

Create named volume

```
docker volume create mysql_data
```

Launch MySQL

```
docker run -dit \
```

```
--name mysql_db \  
--network student_net \  
-e MYSQL_ROOT_PASSWORD=root123 \  
-e MYSQL_DATABASE=studentdb \  
-e MYSQL_USER=student \  
-e MYSQL_PASSWORD=student123 \  
-v mysql_data:/var/lib/mysql \  
mysql:8
```

```
# Verify MySQL
```

```
docker ps
```

```
# Create table and insert data
```

```
docker exec -it mysql_db mysql -u root -proot123
```

```
USE studentdb;
```

```
CREATE TABLE student(  
prn VARCHAR(20),  
name VARCHAR(50)  
);
```

```
INSERT INTO student VALUES  
( '260240123060','ADI');
```

```
SELECT * FROM student;
```

Launch client container

```
docker run -dit --name app_client --network student_net ubuntu bash
```

Install mysql client

```
docker exec -it app_client bash
```

```
apt update -y
```

```
apt install mysql-client -y
```

```
mysql -h mysql_db -u student -pstudent123 studentdb
```

```
SELECT * FROM student;
```

Remove and recreate client

```
docker rm -f app_client
```

```
docker run -dit \
```

```
--name app_client \
```

```
--network student_net \
```

```
ubuntu bash
```

Remove MySQL container only

```
docker rm -f mysql_db
```

Recover MySQL using same volume

```
docker run -dit \
```

```
--name mysql_db_new \
```

```
--network student_net \
```

```
-e MYSQL_ROOT_PASSWORD=root123 \
```

```
-e MYSQL_DATABASE=studentdb \
```

```
-e MYSQL_USER=student \
```

```
-e MYSQL_PASSWORD=student123 \
```

```
-v mysql_data:/var/lib/mysql \
```

```
mysql:8
```

```
# Verify data recovered
```

```
docker exec -it mysql_db_new mysql -u root -proot123
```

```
USE studentdb;
```

```
SELECT * FROM student;
```

```
# Network Details
```

```
docker network inspect student_net
```

```
# Volume Details
```

```
docker volume inspect mysql_data
```

```
# Running Containers
```

```
docker ps
```

Q9) Multi-Tier Application Deployment with Swarm Networking and Scaling

Create a Docker Swarm cluster and deploy a two-tier application consisting of an **Nginx frontend service** and a **MySQL backend service**. Create an overlay network named **app_net** and attach both services to it. Configure the MySQL service using environment variables and store its data in a Docker volume. Deploy the frontend service with **3 replicas** and publish it on port **8080**. Verify communication between frontend and backend services through the overlay network. Scale the frontend service to **6 replicas** and verify task distribution across nodes. Inspect the overlay network, services, tasks, and volumes. Finally, remove the services, volume, and network, and leave the swarm.

```
# Initialize Swarm
docker swarm init
# Create Overlay Network
docker network create -d overlay app_net
# Create Volume
docker volume create mysql_vol
# Deploy MySQL Service
docker service create \
--name mysql_service \
--network app_net \
--mount type=volume,source=mysql_vol,target=/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=root123 \
-e MYSQL_DATABASE=studentdb \
mysql:8
# Deploy Nginx Frontend with 3 Replicas
docker service create \
--name nginx_service \
--replicas 3 \
--network app_net \
-p 8080:80 \
nginx
# Verify Services
docker service ls
# Verify Tasks
docker service ps nginx_service
docker service ps mysql_service
# Inspect Network
docker network inspect app_net
# Scale Frontend to 6 Replicas
```

```
docker service scale nginx_service=6
# Verify Scaling
docker service ps nginx_service
# Access Frontend
curl http://localhost:8080
# Volume Details
docker volume inspect mysql_vol
# Remove Services
docker service rm nginx_service
docker service rm mysql_service
# Remove Volume
docker volume rm mysql_vol
# Remove Overlay Network
docker network rm app_net
# Leave Swarm
docker swarm leave --force
```

Verification Commands

```
docker service ls
docker network ls
docker volume ls
docker ps
```

Q10) Rolling Update, Rollback and Node Maintenance

Create a Docker Swarm cluster and deploy an Nginx service named **webapp** with **5 replicas**. Configure the service with an update policy that updates one container at a time with a delay between updates. Perform a rolling update from **nginx:latest** to **nginx:alpine** and monitor the update progress. Simulate a failure by updating the service to a non-existing image and observe the failed deployment. Perform a rollback to the previous working version. Next, place one worker node into **Drain** mode and verify that all service tasks are rescheduled to active nodes. Return the node to **Active** mode and verify cluster health. Display service history, update status, rollback status, and node availability information.

Initialize Swarm

```
docker swarm init
```

Create Service with 5 Replicas

```
docker service create \
```

```
--name webapp \
```

```
--replicas 5 \
```

```
--update-parallelism 1 \
```

```
--update-delay 10s \
```

```
-p 8080:80 \
```

```
nginx:latest
```

Verify Service

```
docker service ls
```

```
docker service ps webapp
```

Rolling Update to nginx:alpine

```
docker service update \
```

```
--image nginx:alpine \
```

```
webapp
```


Monitor Update

docker service ps webapp

docker service inspect webapp --pretty

Simulate Failure

docker service update \

--image nginx:notfound \

webapp

Check Failed Update

docker service ps webapp

Rollback to Previous Working Version

docker service rollback webapp

Verify Rollback

docker service ps webapp

docker service inspect webapp --pretty

View Nodes

docker node ls

Drain Worker Node

docker node update --availability drain worker1

Verify Tasks Rescheduled

docker service ps webapp

docker node ls

Return Node to Active

docker node update --availability active worker1

Verify Cluster Health

docker node ls

docker service ls

Service History and Status

docker service inspect webapp --pretty

docker service ps webapp

docker node ls

Q11) Swarm Stack Deployment with Volumes, Secrets and Recovery

Create a Docker Swarm cluster and deploy a stack named **student_stack** using a Compose file. The stack must contain:

- A MySQL service
- A phpMyAdmin service
- A custom web service

Store database credentials using **Docker Secrets**. Use a named volume for database persistence. Create an overlay network connecting all services. Verify that phpMyAdmin can communicate with the database and that the web service can access database records.

Scale the web service from **2 replicas to 5 replicas** and verify load distribution. Remove the MySQL service container manually and verify that Swarm automatically recreates it. Verify that the database data remains intact because of the persistent volume. Inspect stack services, networks, volumes, secrets, and tasks. Finally, remove the entire stack and verify that all associated services are removed successfully.

Initialize Swarm

docker swarm init

Create Overlay Network

docker network create \

```
-d overlay \
```

```
student_net
```

```
# Create Volume
```

```
docker volume create mysql_data
```

```
# Create Secrets
```

```
echo "root123" | docker secret create mysql_root_password -
```

```
echo "student123" | docker secret create mysql_password -
```

docker-compose.yml

```
version: "3.8"
```

```
services:
```

```
mysql:
```

```
image: mysql:8
```

```
networks:
```

```
- student_net
```

```
volumes:
```

```
- mysql_data:/var/lib/mysql
```

```
environment:
```

```
MYSQL_DATABASE: studentdb
```

```
MYSQL_USER: student
```

```
MYSQL_PASSWORD_FILE: /run/secrets/mysql_password
```

```
MYSQL_ROOT_PASSWORD_FILE: /run/secrets/mysql_root_password
```

secrets:

- mysql_root_password
- mysql_password

phpmyadmin:

image: phpmyadmin/phpmyadmin

ports:

- "8080:80"

networks:

- student_net

environment:

PMA_HOST: mysql

web:

image: nginx

ports:

- "8081:80"

networks:

- student_net

networks:

student_net:

external: true

volumes:

mysql_data:

secrets:

mysql_root_password:

external: true

mysql_password:

external: true

Deploy Stack

docker stack deploy \

-c docker-compose.yml \

student_stack

Verify Stack

docker stack ls

docker stack services student_stack

Verify Running Containers

docker service ls

Verify Network

docker network inspect student_net

Verify Volume

docker volume inspect mysql_data

Open phpMyAdmin

http://<PUBLIC-IP>:8080

Login:

Server : mysql

User : root

Password : root123

Verify Web Service

http://<PUBLIC-IP>:8081

View Secrets

docker secret ls

Remove Stack

docker stack rm student_stack

Redeploy Stack (Recovery Test)

docker stack deploy \

-c docker-compose.yml \

student_stack

Verify Data Still Exists

docker volume inspect mysql_data

Verification Commands

docker stack ls

docker stack services student_stack

docker service ls

docker network ls

docker volume ls

docker secret ls